MACHINE LEARNING

>:_ FACULTAD DE INGENIERÍA

Universidad Rafael Landívar
Identidad Jesuita en Guatemala

POSGRADOS LANDÍVAR

Laboratorio 1.2

Limpieza y Preparación de Datasets

Importación de librerías

In [1]:

```
%matplotlib inline

import matplotlib

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Carga de datasets

Cargue los dos datasets utilizando la función `pd.read_csv()`
Utilice las opciones `sep = "|"` **y** `header = 0`

`sep` ***indica como estan separadas las columnas***
`header` **si la primera fila del dataset es el nombre de las columna. Podemos pasar como parámetro 0 o no indicar, por defecto, ya que ambar infiere que el header es la primera línea**

In [3]:

```
clientes = pd.read_csv("[redacted]/clientes.txt", sep = "|", header = 0)
transacciones = pd.read_csv("/[redacted]/transacciones.txt", sep = "|", header = 0)
```

In [5]:

```
clientes.head()
```

Out[5]:

	nombre	apellido	tarjeta	fecha_nacimiento
0	Rosa	Barrientos	1	1984-12-19
1	Juan	Media	2	1954-01-08
2	José	Coronado	3	1984-08-25
3	Mario	Flores	4	1982-11-11
4	Margarita	Muñoz	5	1984-04-11

Determine que tipos de columnas tiene y modifique el tipo si alguna columna no tiene el formato correcto.

In [6]:

```
clientes.dtypes
```

Out[6]:

```
nombre               object
apellido             object
tarjeta               int64
fecha_nacimiento     object
dtype: object
```

In [7]:

```
transacciones.dtypes
```

Out[7]:

```
tarjeta     float64
monto       float64
comercio     object
fecha        object
tipo         object
dtype: object
```

Necesitamos cambiar las columnas que son fechas de formato object a datime. Esto lo hacemos con la función `astype`

In [8]:

```
clientes['fecha_nacimiento'] = clientes['fecha_nacimiento'].astype('datetime64[ns]')
```

In [9]:

```
transacciones['fecha'] = transacciones['fecha'].astype('datetime64[ns]')
```

Verificamos nuevamente los tipos de dato

In [10]:

```
clientes.dtypes
```

Out[10]:

```
nombre                      object
apellido                    object
tarjeta                      int64
fecha_nacimiento    datetime64[ns]
dtype: object
```

In [11]:

```
transacciones.dtypes
```

Out[11]:

```
tarjeta             float64
monto               float64
comercio             object
fecha       datetime64[ns]
tipo                 object
dtype: object
```

Explore la tabla transacciones y responda las siguientes preguntas:

1. ¿Todas las tarjetas que se encuentran en la tabla de clientes tienen transacciones?

Solución --> Utilizaremos la función `unique` para obtener la cantidad de tarjetas únicas que se encuentran en el dataset. Luego, veremos la cantidad de registros con la función `shape`. La forma de leer el output de esta función es el siguiente (filas, columnas)

In [12]:

```
tarjetas_unicas = clientes['tarjeta'].unique()
len(tarjetas_unicas)
```

Out[12]:

```
110
```

In [13]:

```
tarjetas_transacciones_unicas = transacciones['tarjeta'].unique()
len(tarjetas_transacciones_unicas)
```

Out[13]:

```
101
```

Los lengths de ambas tablas para el unique de tarjeta nos da valores diferentes, por lo que no todas las tarjetas que se encuentran en la tabla de clientes tienen transacciones

2. ¿Cuál es la tarjeta que tiene más tranascciones? ¿Cuál tiene menos?

Solución --> Utilizaremos la función `value_counts`, le agregaremos los parámetros de `ascending = False` para que nos retorne de forma descendente y `dropna = False` para que no excluya a los valores NaN. Utilizaremos `value_counts` en lugar de `groupby` ya que `groupby` no reconoce los NaN a la hora de hacer el count

In [14]:

```
qty_transacciones = transacciones['tarjeta'].value_counts(ascending=False, dropna=False)
qty_transacciones
```

Out[14]:

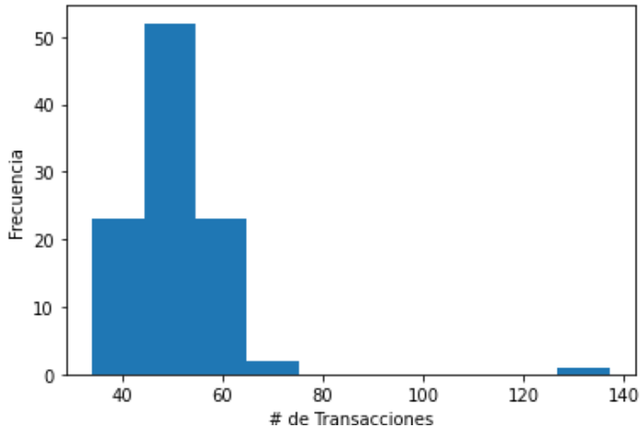
```
NaN      137
96.0      66
100.0     65
74.0      62
57.0      62
         ...
6.0       39
56.0      39
67.0      39
5.0       37
34.0      34
Name: tarjeta, Length: 101, dtype: int64
```

La tarjeta que más transacciones tiene es la *NaN* con 137 transacciones, luego va la *96* con 66. La tarjeta con menos transacciones es la *34* con 34.

3. Grafique la cantidad de transacciones por tarjeta y determine: ¿Hay algo extraño? Si sí, solucionelo.

In [15]:

```
plt.xlabel('# de Transacciones')
plt.ylabel('Frecuencia')
_ = plt.hist(qty_transacciones, bins = 10)
```



Con el histograma podemos ver que tenemos un valor atipico atribuido a que tenemos tarjeta con valor NA

Debemos filtrar todas las tarjetas que su valor sea NA por que no las podemos relacionar y por que es el valor atipico de 140 transaccione

In [16]:

```
transacciones_filt = transacciones[transacciones['tarjeta'].notnull()]
```

In [17]:

```
qty_transacciones_filt = transacciones_filt['tarjeta'].value_counts(ascending=False, drop
na=False)
qty_transacciones_filt
```

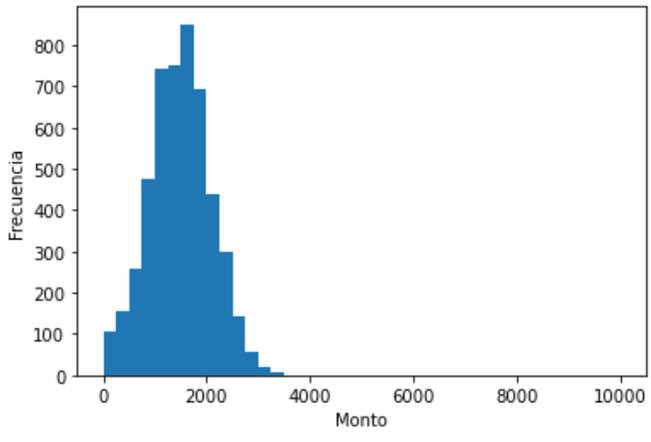
Out[17]:

```
96.0     66
100.0    65
74.0     62
57.0     62
81.0     61
         ..
67.0     39
56.0     39
6.0      39
5.0      37
34.0     34
Name: tarjeta, Length: 100, dtype: int64
```

4. ¿Tiene outliers y datos extraños en la columna monto?

In [26]:

```
plt.xlabel('Monto')
plt.ylabel('Frecuencia')
_ = plt.hist(transacciones_filt['monto'], bins = 40)
```



In [27]:

```
transacciones_filt[transacciones_filt['monto'] > 4000]
```

Out[27]:

	tarjeta	monto	comercio	fecha	tipo
2281	57.0	4094.69	Gasolineras	2020-02-07	transaccion
4118	37.0	10000.00	Supermercados	2020-05-04	reconciliacion

Sí, aparece una transacción de aproximadamente 10,0000 y por eso la curva se ve tan sesgada

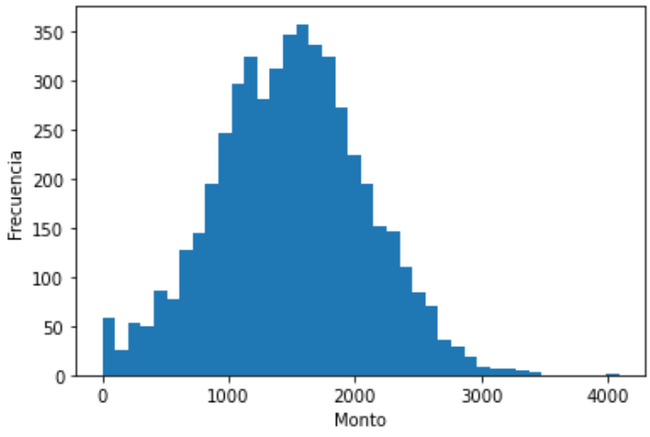
Filtramos la transaccion de 10,000 por ser un outlier

In [28]:

```
transacciones_filt = transacciones_filt[transacciones_filt['monto'] < 7000]
```

In [29]:

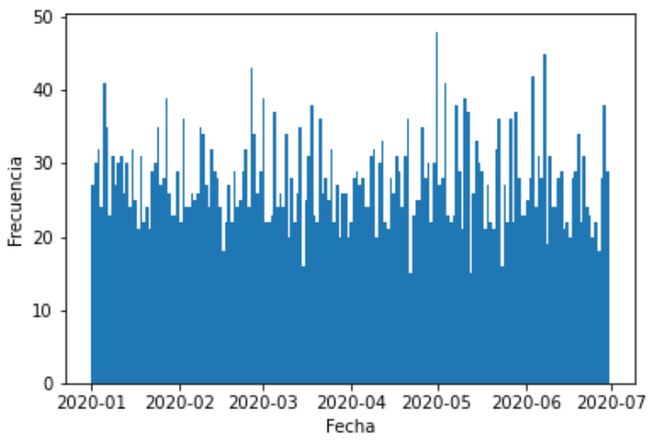
```
plt.xlabel('Monto')
plt.ylabel('Frecuencia')
_ = plt.hist(transacciones_filt['monto'], bins = 40)
```



5. ¿Qué cantidad de transacciones hay por fecha? ¿Existe algun tipo de estacionalidad?

In [30]:

```
plt.xlabel('Fecha')
plt.ylabel('Frecuencia')
_ = plt.hist(transacciones_filt['fecha'], bins = 182)
```



Al graficar por conteo de las fechas, no podemos observar algún tipo de estacionalidad. Se requerirá separar la fecha por los meses del día y por mes para obtener más información

In [32]:

```
transacciones_filt['diaSemana'] = transacciones_filt['fecha'].dt.day_name()
```

In [33]:

```
transacciones_filt['weekday'] = transacciones_filt['fecha'].dt.dayofweek
```

In [34]:

```
transacciones_filt['diaMes'] = transacciones_filt['fecha'].dt.day
```

In [35]:

```
transacciones_filt['mes'] = transacciones_filt['fecha'].dt.month
```

In [36]:

```
transacciones_filt.head()
```

Out[36]:

	tarjeta	monto	comercio	fecha	tipo	diaSemana	weekday	diaMes	mes
0	90.0	2067.31	Gasolineras	2020-01-05	transaccion	Sunday	6	5	1
1	67.0	1871.79	Supermercados	2020-02-16	transaccion	Sunday	6	16	2
2	1.0	2171.59	Supermercados	2020-06-04	transaccion	Thursday	3	4	6
3	90.0	692.46	Gasolineras	2020-01-14	transaccion	Tuesday	1	14	1
4	31.0	1463.87	Tiendas por Departamento	2020-05-28	transaccion	Thursday	3	28	5

In [37]:

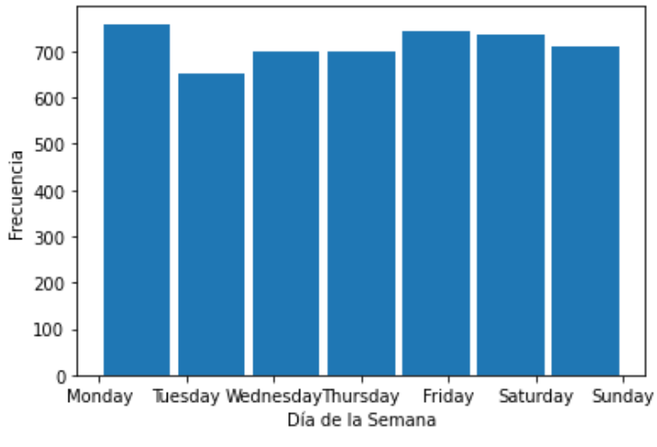
```
qty_dias = transacciones_filt[['diaSemana']].value_counts(ascending=False, dropna=False)
qty_dias
```

Out[37]:

```
diaSemana
Monday       759
Friday       742
Saturday     737
Sunday       709
Thursday     701
Wednesday    701
Tuesday      650
dtype: int64
```

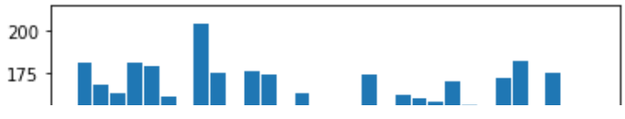
In [38]:

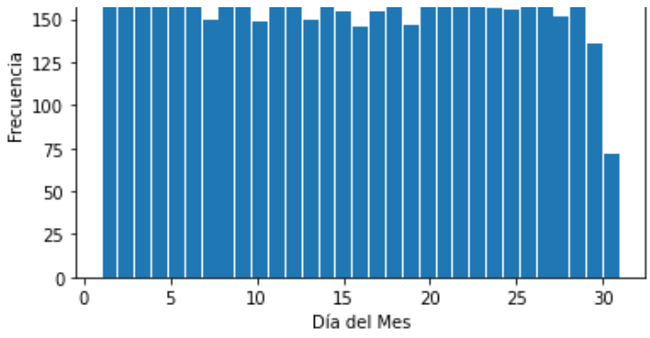
```
plt.xlabel('Día de la Semana')
plt.ylabel('Frecuencia')
_ = plt.hist(transacciones_filt.sort_values('weekday')['diaSemana'], bins = 7, rwidth=0.
9)
```



In [39]:

```
plt.xlabel('Día del Mes')
plt.ylabel('Frecuencia')
_ = plt.hist(transacciones_filt['diaMes'], bins = 31, rwidth=0.9)
```





6. ¿Qué indica la columna tipo? Encuentre cuántas veces aparece cada tipo de transaccion y qué ocurre cuando aparece el tipo "reconciliacion".

In [40]:

```
qty_tipo = transacciones_filt['tipo'].value_counts(ascending=False, dropna=False)
qty_tipo
```

Out[40]:

```
transaccion       4961
reconciliacion      38
Name: tipo, dtype: int64
```

In [41]:

```
reconciliacion = transacciones_filt[transacciones_filt['tipo'] == 'reconciliacion']
reconciliacion
```

Out[41]:

	tarjeta	monto	comercio	fecha	tipo	diaSemana	weekday	diaMes	mes
218	44.0	0.0	Supermercados	2020-02-02	reconciliacion	Sunday	6	2	2
309	36.0	0.0	Restaurantes	2020-06-29	reconciliacion	Monday	0	29	6
381	68.0	0.0	Restaurantes	2020-02-29	reconciliacion	Saturday	5	29	2
465	31.0	0.0	Tiendas por Departamento	2020-04-27	reconciliacion	Monday	0	27	4
590	54.0	0.0	Supermercados	2020-05-14	reconciliacion	Thursday	3	14	5
593	37.0	0.0	Comercio General	2020-01-23	reconciliacion	Thursday	3	23	1
615	32.0	0.0	Abarroterias	2020-06-20	reconciliacion	Saturday	5	20	6
1297	62.0	0.0	Farmacias	2020-01-20	reconciliacion	Monday	0	20	1
1500	96.0	0.0	Farmacias	2020-03-04	reconciliacion	Wednesday	2	4	3
1646	93.0	0.0	Supermercados	2020-02-16	reconciliacion	Sunday	6	16	2
1648	41.0	0.0	Supermercados	2020-04-26	reconciliacion	Sunday	6	26	4
1670	31.0	0.0	Comercio General	2020-01-18	reconciliacion	Saturday	5	18	1
1724	75.0	0.0	Supermercados	2020-01-26	reconciliacion	Sunday	6	26	1
1887	96.0	0.0	Ropa	2020-04-09	reconciliacion	Thursday	3	9	4
2117	92.0	0.0	Restaurantes	2020-04-24	reconciliacion	Friday	4	24	4
2196	89.0	0.0	Restaurantes	2020-01-17	reconciliacion	Friday	4	17	1
2202	32.0	0.0	Restaurantes	2020-05-11	reconciliacion	Monday	0	11	5
2338	20.0	0.0	Farmacias	2020-05-22	reconciliacion	Friday	4	22	5
2384	99.0	0.0	Ropa	2020-03-12	reconciliacion	Thursday	3	12	3
2386	42.0	0.0	Supermercados	2020-03-02	reconciliacion	Monday	0	2	3
2433	36.0	0.0	Restaurantes	2020-05-29	reconciliacion	Friday	4	29	5
2454	47.0	0.0	Gasolineras	2020-06-09	reconciliacion	Tuesday	1	9	6

	tarjeta	monto	comercio	fecha	tipo	diaSemana	weekday	diaMes	mes
2629	63.0	0.0	Supermercados	2020-04-20	reconciliacion	Monday	0	20	4
2742	73.0	0.0	Gasolineras	2020-05-16	reconciliacion	Saturday	5	16	5
2922	30.0	0.0	Supermercados	2020-05-06	reconciliacion	Wednesday	2	6	5
2993	66.0	0.0	Restaurantes	2020-05-31	reconciliacion	Sunday	6	31	5
3000	43.0	0.0	Supermercados	2020-03-21	reconciliacion	Saturday	5	21	3
3025	85.0	0.0	Farmacias	2020-02-10	reconciliacion	Monday	0	10	2
3173	77.0	0.0	Comercio General	2020-05-01	reconciliacion	Friday	4	1	5
3337	20.0	0.0	Restaurantes	2020-06-19	reconciliacion	Friday	4	19	6
3397	57.0	0.0	Supermercados	2020-05-04	reconciliacion	Monday	0	4	5
3615	52.0	0.0	Restaurantes	2020-06-02	reconciliacion	Tuesday	1	2	6
3889	100.0	0.0	Supermercados	2020-06-24	reconciliacion	Wednesday	2	24	6
3932	9.0	0.0	Farmacias	2020-03-02	reconciliacion	Monday	0	2	3
3940	6.0	0.0	Supermercados	2020-05-03	reconciliacion	Sunday	6	3	5
4227	38.0	0.0	Restaurantes	2020-01-27	reconciliacion	Monday	0	27	1
4346	22.0	0.0	Tiendas por Departamento	2020-06-25	reconciliacion	Thursday	3	25	6
4877	45.0	0.0	Restaurantes	2020-02-23	reconciliacion	Sunday	6	23	2

In [43]:

```
reconciliacion[reconciliacion['monto'] > 0]
```

Out[43]:

	tarjeta	monto	comercio	fecha	tipo	diaSemana	weekday	diaMes	mes

Se puede determinar analizando la tabla que cuando el tipo es *reconciliacion* siempre tiene un monto de 0, por lo que este tipo de transaccion no nos interesa y lo podemos filtrar

In [44]:

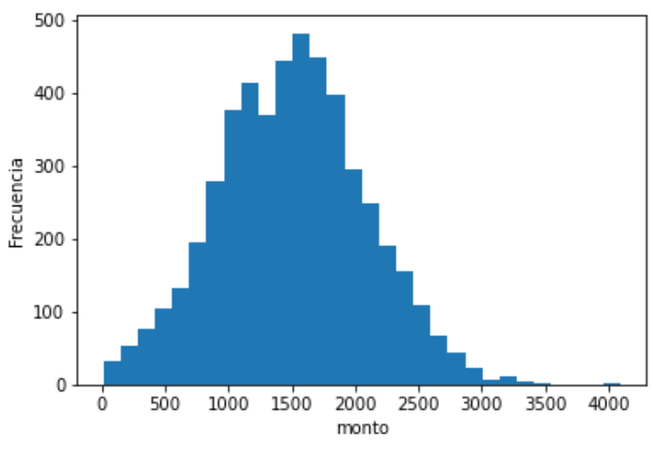
```
transacciones_filt = transacciones_filt[transacciones_filt['tipo'] != 'reconciliacion']
transacciones_filt['tipo'].unique()
```

Out[44]:

```
array(['transaccion'], dtype=object)
```

In [45]:

```
plt.xlabel('monto')
plt.ylabel('Frecuencia')
_ = plt.hist(transacciones_filt['monto'], bins = 30)
```



La funcion de boxplot me permite graficar para ver si existen outliers dentro de mi dataset en funcion de la

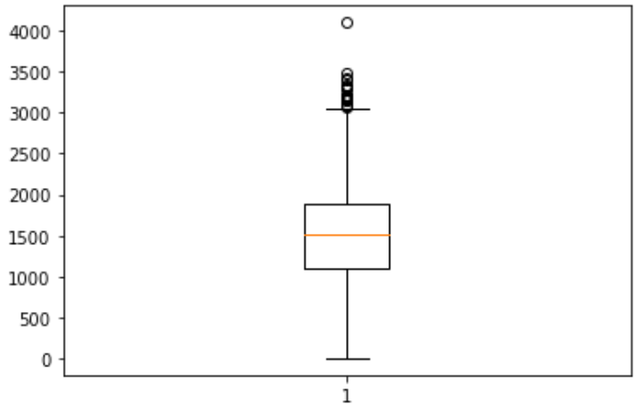
columna monto

In [50]:

```
plt.boxplot(transacciones_filt['monto'])
```

Out[50]:

```
{'whiskers': [<matplotlib.lines.Line2D at 0x7f803805cd90>,
  <matplotlib.lines.Line2D at 0x7f803806f0a0>],
 'caps': [<matplotlib.lines.Line2D at 0x7f803806f370>,
  <matplotlib.lines.Line2D at 0x7f803806f640>],
 'boxes': [<matplotlib.lines.Line2D at 0x7f803805ca60>],
 'medians': [<matplotlib.lines.Line2D at 0x7f803806f910>],
 'fliers': [<matplotlib.lines.Line2D at 0x7f803806fbe0>],
 'means': []}
```

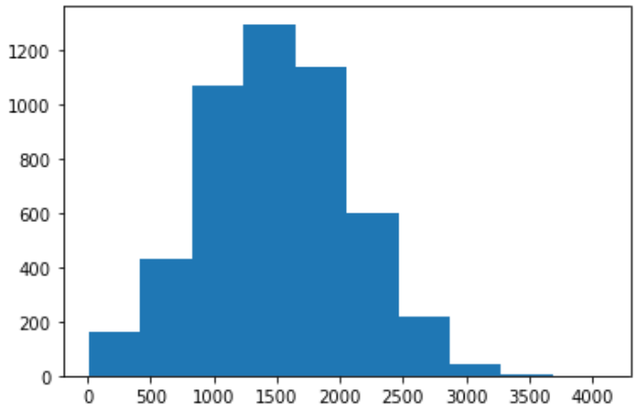


In [51]:

```
plt.hist(transacciones_filt['monto'])
```

Out[51]:

```
(array([1.610e+02, 4.330e+02, 1.069e+03, 1.295e+03, 1.140e+03, 5.960e+02,
        2.190e+02, 4.000e+01, 7.000e+00, 1.000e+00]),
 array([  10.91 ,  419.288,  827.666, 1236.044, 1644.422, 2052.8  ,
        2461.178, 2869.556, 3277.934, 3686.312, 4094.69 ]),
 <BarContainer object of 10 artists>)
```



7. Para la tabla clientes obtenga los años de vida de sus clientes y grafiquelo

In [52]:

```
clientes.head()
```

Out[52]:

	nombre	apellido	tarjeta	fecha_nacimiento
0	Rosa	Barrientos	1	1984-12-19

	nombre	apellido	tarjeta	fecha_nacimiento
2	José	Coronado	3	1984-08-25
3	Mario	Flores	4	1982-11-11
4	Margarita	Muñoz	5	1984-04-11

In [53]:

```
clientes["edad"] = clientes["fecha_nacimiento"].apply(lambda x : (pd.datetime.now() - x)
)
clientes["edad"] = clientes["edad"].astype("timedelta64[Y]")
```

```
/var/folders/8c/150ftmgx56l0y53sz_qry6ym0000gp/T/ipykernel_66308/532557761.py:1: FutureWa
rning: The pandas.datetime class is deprecated and will be removed from pandas in a futur
e version. Import from datetime module instead.
  clientes["edad"] = clientes["fecha_nacimiento"].apply(lambda x : (pd.datetime.now() - x
))
```

In [54]:

```
clientes.head()
```

Out[54]:

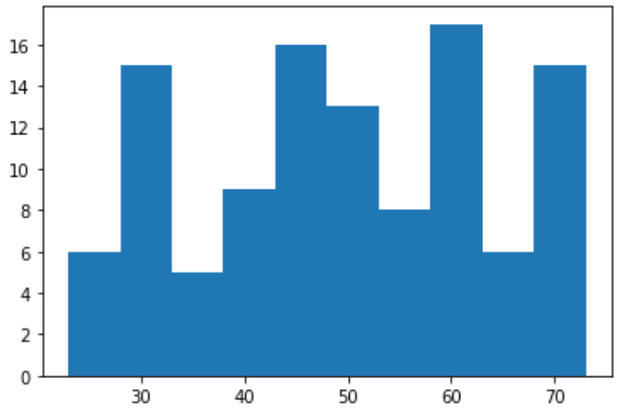
	nombre	apellido	tarjeta	fecha_nacimiento	edad
0	Rosa	Barrientos	1	1984-12-19	38.0
1	Juan	Media	2	1954-01-08	69.0
2	José	Coronado	3	1984-08-25	38.0
3	Mario	Flores	4	1982-11-11	40.0
4	Margarita	Muñoz	5	1984-04-11	38.0

In [55]:

```
plt.hist(clientes['edad'])
```

Out[55]:

```
(array([ 6., 15.,  5.,  9., 16., 13.,  8., 17.,  6., 15.]),
 array([23., 28., 33., 38., 43., 48., 53., 58., 63., 68., 73.]),
 <BarContainer object of 10 artists>)
```



8. Usted desea hacer un analisis mensual, por lo que en su tabla base su unidad de analisis debe ser la combinacion de cliente mes

In [56]:

```
transacciones_filt['fecha'].min()
```

Out[56]:

```
Timestamp('2020-01-01 00:00:00')
```

In [57]:

```
transacciones_filt['fecha'].max()
```

Out[57]:

```
Timestamp('2020-06-30 00:00:00')
```

Para obtener la unidad de analisis, obtenemos la combinación de año-mes que nos interesan. Son exactamente seis meses utilizando las funciones de max y min anteriores

In [59]:

```
fechas = transacciones_filt['fecha'].unique()
fechas = pd.DataFrame(fechas, columns = ['fecha'])
```

In [45]:

```
fechas
```

Out[45]:

	fecha
0	2020-01-05
1	2020-02-16
2	2020-06-04
3	2020-01-14
4	2020-05-28
...	...
177	2020-01-13
178	2020-02-20
179	2020-02-21
180	2020-06-26
181	2020-05-24

182 rows × 1 columns

Para poder extraer solo los 6 meses que nos interesan y tener 6 fechas (1 por mes), será necesario extraer el mes y el año de cada fecha en esos 6 meses y pegarle un 01. De esta forma, obtendremos fechas con un formato como el siguiente: *YYYY/MM/01*

In [60]:

```
fechas['year'] = pd.DatetimeIndex(fechas['fecha']).year
fechas['month'] = pd.DatetimeIndex(fechas['fecha']).month

fechas["fechaAM"] = fechas["year"].astype(str) + "/" +  "0" + fechas["month"].astype(str
) + "/" + "01"
```

In [61]:

```
fechas.head()
```

Out[61]:

	fecha	year	month	fechaAM
0	2020-01-05	2020	1	2020/01/01
1	2020-02-16	2020	2	2020/02/01
2	2020-06-04	2020	6	2020/06/01

	fecha	year	month	fechaAM
3	2020-01-14	2020	1	2020/01/01
4	2020-05-28	2020	5	2020/05/01

In [62]:

```
fechas = fechas['fechaAM'].unique()
fechas
```

Out[62]:

```
array(['2020/01/01', '2020/02/01', '2020/06/01', '2020/05/01',
       '2020/04/01', '2020/03/01'], dtype=object)
```

Una vez tenemos los 6 meses que vamos a analizar, debemos realizar un merge con los clientes para que cada cliente se repita 6 veces (una vez por cada mes que analizamos). Sin embargo, primero debemos entender si cada observación en la tabla de clientes está expresada en una línea por cliente o una línea por tarjeta

In [66]:

```
clientes.shape
```

Out[66]:

```
(110, 5)
```

In [67]:

```
len(clientes['tarjeta'].unique())
```

Out[67]:

```
110
```

In [70]:

```
len(clientes.groupby(["nombre", "apellido","fecha_nacimiento"]).count())
```

Out[70]:

```
110
```

In [68]:

```
len(clientes.groupby(["nombre", "apellido"]).count())
```

Out[68]:

```
100
```

In [69]:

```
nombres_gb = pd.DataFrame(clientes.groupby(['nombre', 'apellido']).size().reset_index(na
me='counts'))
nombres_gb[nombres_gb.counts > 1]
```

Out[69]:

	nombre	apellido	counts
14	Andrés	Rivera	2
15	Andrés	Ruíz	2
43	Gustavo	Ruíz	2
53	José	Coronado	2
56	Juan	Calderón	2
61	Juan	Perez	3
70	Luis	Figueroa	2
71	Luis	Martinez	2

	nombre	apellido	counts
79	Maria	Juarez	2

In [71]:

```
tarjetas_gb = pd.DataFrame(clientes.groupby(['tarjeta']).size().reset_index(name='counts
'))
tarjetas_gb[tarjetas_gb.counts > 1]
```

Out[71]:

tarjeta	counts

Hemos logrado determinar que en la tabla de cliente, la combinación nombre-apellido no es suficiente para definir la unidad de análisis, ya que hay personas con el mismo nombre y apellido pero con diferente fecha de nacimiento.

In [72]:

```
fechas = pd.DataFrame(fechas, columns=['fechaMA'])
fechas.head()
```

Out[72]:

	fechaMA
0	2020/01/01
1	2020/02/01
2	2020/06/01
3	2020/05/01
4	2020/04/01

In [73]:

```
clientes['tmp'] = 1
fechas['tmp'] = 1
```

In [74]:

```
fechas.head()
```

Out[74]:

	fechaMA	tmp
0	2020/01/01	1
1	2020/02/01	1
2	2020/06/01	1
3	2020/05/01	1
4	2020/04/01	1

In [75]:

```
clientes.shape
```

Out[75]:

```
(110, 6)
```

In [76]:

```
tabla_base = pd.merge(clientes, fechas, on=['tmp'])
tabla_base = tabla_base.drop('tmp', axis=1)
tabla_base.head(6)
```

Out[76]:

	nombre	apellido	tarjeta	fecha_nacimiento	edad	fechaMA
0	Rosa	Barrientos	1	1984-12-19	38.0	2020/01/01
1	Rosa	Barrientos	1	1984-12-19	38.0	2020/02/01
2	Rosa	Barrientos	1	1984-12-19	38.0	2020/06/01
3	Rosa	Barrientos	1	1984-12-19	38.0	2020/05/01
4	Rosa	Barrientos	1	1984-12-19	38.0	2020/04/01
5	Rosa	Barrientos	1	1984-12-19	38.0	2020/03/01

In [77]:

```
tabla_base.shape
```

Out[77]:

```
(660, 6)
```

In [78]:

```
tabla_base.rename(columns = {'fechaMA':'mes_anio'}, inplace = True)
tabla_base.head()
```

Out[78]:

	nombre	apellido	tarjeta	fecha_nacimiento	edad	mes_anio
0	Rosa	Barrientos	1	1984-12-19	38.0	2020/01/01
1	Rosa	Barrientos	1	1984-12-19	38.0	2020/02/01
2	Rosa	Barrientos	1	1984-12-19	38.0	2020/06/01
3	Rosa	Barrientos	1	1984-12-19	38.0	2020/05/01
4	Rosa	Barrientos	1	1984-12-19	38.0	2020/04/01

Para calcular la edad de forma correcta, tenemos que encontrar la diferencia de la fecha de nacimiento contra el mes que se esta analizando y asi que cambie con cada mes que pase

In [79]:

```
tabla_base['mes_anio'] = tabla_base['mes_anio'].apply(pd.to_datetime)
tabla_base = tabla_base.assign(edad = lambda x : (x['mes_anio'] - x['fecha_nacimiento'])
.dt.days / 365)
tabla_base.head(6)
```

Out[79]:

	nombre	apellido	tarjeta	fecha_nacimiento	edad	mes_anio
0	Rosa	Barrientos	1	1984-12-19	35.057534	2020-01-01
1	Rosa	Barrientos	1	1984-12-19	35.142466	2020-02-01
2	Rosa	Barrientos	1	1984-12-19	35.473973	2020-06-01
3	Rosa	Barrientos	1	1984-12-19	35.389041	2020-05-01
4	Rosa	Barrientos	1	1984-12-19	35.306849	2020-04-01
5	Rosa	Barrientos	1	1984-12-19	35.221918	2020-03-01

9. Una vez haya creado su tabla base, calcule, con la tabla de transacciones, las siguientes variables de forma mensual promedio:

- monto por transaccion promedio
- cantidad de transacciones promedio

- **sumatoria de monto promedio**

Para calcular las variables primero tenemos que definir la uniad de analisis mes-cliente

In [81]:

```
transacciones_filt['year'] = pd.DatetimeIndex(transacciones_filt['fecha']).year

transacciones_filt["mes_anio"] = transacciones_filt["year"].astype(str) + "/" +  "0" + t
ransacciones_filt["mes"].astype(str) + "/" + "01"
transacciones_filt['mes_anio'] = transacciones_filt['mes_anio'].apply(pd.to_datetime)

transacciones_filt = transacciones_filt.drop(columns=['year', "diaSemana", "diaMes", "mes
"])
transacciones_filt.head()
```

Out[81]:

	tarjeta	monto	comercio	fecha	tipo	weekday	mes_anio
0	90.0	2067.31	Gasolineras	2020-01-05	transaccion	6	2020-01-01
1	67.0	1871.79	Supermercados	2020-02-16	transaccion	6	2020-02-01
2	1.0	2171.59	Supermercados	2020-06-04	transaccion	3	2020-06-01
3	90.0	692.46	Gasolineras	2020-01-14	transaccion	1	2020-01-01
4	31.0	1463.87	Tiendas por Departamento	2020-05-28	transaccion	3	2020-05-01

Las variables las calculamos a partir de la unidad de analisis definida en las lineas anteriores

In [82]:

```
variables_mensual = (transacciones_filt.groupby(['tarjeta', "mes_anio"])
    .agg({'comercio':'count', 'monto': 'sum'})
    .reset_index()
    .rename(columns={'comercio':'conteo_mensual', 'monto': 'sumatoria_mensual'})
)

variables_mensual = variables_mensual.assign(promedio_monto_mensual = lambda x : x['suma
toria_mensual'] / x['conteo_mensual'])

variables_mensual.head()
```

Out[82]:

	tarjeta	mes_anio	conteo_mensual	sumatoria_mensual	promedio_monto_mensual
0	1.0	2020-01-01	6	7415.41	1235.901667
1	1.0	2020-02-01	5	9578.51	1915.702000
2	1.0	2020-03-01	9	14300.67	1588.963333
3	1.0	2020-04-01	13	22509.28	1731.483077
4	1.0	2020-05-01	4	6377.68	1594.420000

Para comprobar que el monto promedio de el codigo anterior si funciona para la tarteja 1 en el mes de enero

In [83]:

```
(
    transacciones_filt.loc[
        (transacciones_filt['tarjeta'] == 1) &
        (transacciones_filt['fecha'] <= "2020-01-31 00:00:00")
    ].groupby("tarjeta")['monto'].mean()
)
```

Out[83]:

```
tarjeta
1.0    1235.901667
Name: monto, dtype: float64
```

In [84]:

```
tabla_base['mes_anio'] = tabla_base['mes_anio'].astype('datetime64[ns]')
```

10. Realice un join para "pegarle" sus variables calculadas a la tabla base

Utilizamos un left join para pegarle las variables calculadas a la tabla base

In [87]:

```
base_vars = tabla_base.merge(variables_mensual, on=['mes_anio', 'tarjeta'], how='left')
base_vars.head()
```

Out[87]:

	nombre	apellido	tarjeta	fecha_nacimiento	edad	mes_anio	conteo_mensual	sumatoria_mensual	promedio_monto
0	Rosa	Barrientos	1	1984-12-19	35.057534	2020-01-01	6.0	7415.41	12
1	Rosa	Barrientos	1	1984-12-19	35.142466	2020-02-01	5.0	9578.51	19
2	Rosa	Barrientos	1	1984-12-19	35.473973	2020-06-01	10.0	16552.09	16
3	Rosa	Barrientos	1	1984-12-19	35.389041	2020-05-01	4.0	6377.68	15
4	Rosa	Barrientos	1	1984-12-19	35.306849	2020-04-01	13.0	22509.28	17

In [90]:

```
len(base_vars[base_vars['conteo_mensual'].isnull()])
```

Out[90]:

```
60
```

Podemos observar que existen 60 observaciones en donde el conteo mensual es de 60. Esto ocurre por que 10 tarjetas no tenían ninguna transaccion en los 6 meses que los analizamos.

In [92]:

```
base_vars[['conteo_mensual', 'sumatoria_mensual','promedio_monto_mensual']] = (
base_vars[['conteo_mensual', 'sumatoria_mensual','promedio_monto_mensual']].fillna(value=
0) )
```

11. Calcule las variables para encontrar que porcentaje de las transacciones se realizan en cada tipo de comercio para cada tarjeta. Ademas, compruebe que sus calculos son correctos. ¿Encuentra algo raro en las variables calculadas de comercio?

Para poder calcular la variable, primero agrupamos por tarjeta, mes_anio y comercio esto nos dice cada cliente en cada mes cuantas transacciones hace en cada comercio

In [94]:

```
variables_comercio = (transacciones_filt.groupby(['tarjeta', "mes_anio", "comercio"],drop
na=False)
    .agg({'tipo':'count'})
    .reset_index()
    .rename(columns={'tipo':'conteo_comercio'})
```

```
)

variables_comercio.head()
```

Out[94]:

	tarjeta	mes_anio	comercio	conteo_comercio
0	1.0	2020-01-01	Farmacias	1
1	1.0	2020-01-01	Restaurantes	1
2	1.0	2020-01-01	Ropa	1
3	1.0	2020-01-01	Supermercados	3
4	1.0	2020-02-01	Gasolineras	1

Luego, agrupamos por tarjeta y mes anio para obtener la cantidad de transacciones en cada combinacion de tarjeta mes_anio

In [93]:

```
conteo_tarjeta = (transacciones_filt.groupby(['tarjeta', "mes_anio"],dropna=False)
    .agg({'tipo':'count'})
    .reset_index()
    .rename(columns={'tipo':'conteo_mensual'})
)

conteo_tarjeta.head()
```

Out[93]:

	tarjeta	mes_anio	conteo_mensual
0	1.0	2020-01-01	6
1	1.0	2020-02-01	5
2	1.0	2020-03-01	9
3	1.0	2020-04-01	13
4	1.0	2020-05-01	4

Unimos el conteo por comercio contra el conteo total para obtener la variable de porcentaje de transacciones realizadas en cada comercio

In [95]:

```
variables_comercio = variables_comercio.merge(conteo_tarjeta, on = ['tarjeta', 'mes_anio'
], how = 'left')
variables_comercio.head()
```

Out[95]:

	tarjeta	mes_anio	comercio	conteo_comercio	conteo_mensual
0	1.0	2020-01-01	Farmacias	1	6
1	1.0	2020-01-01	Restaurantes	1	6
2	1.0	2020-01-01	Ropa	1	6
3	1.0	2020-01-01	Supermercados	3	6
4	1.0	2020-02-01	Gasolineras	1	5

In [96]:

```
variables_comercio = variables_comercio.assign(trans_por = lambda x : x['conteo_comercio
'] / x['conteo_mensual'])
variables_comercio.head()
```

Out[96]:

	tarjeta	mes_anio	comercio	conteo_comercio	conteo_mensual	trans_por
0	1.0	2020-01-01	Farmacias	1	6	0.166667
1	1.0	2020-01-01	Restaurantes	1	6	0.166667
2	1.0	2020-01-01	Ropa	1	6	0.166667
3	1.0	2020-01-01	Supermercados	3	6	0.500000
4	1.0	2020-02-01	Gasolineras	1	5	0.200000

Para comprobar si lo hicimos bien, comparamos la sumatoria de las transacciones porcentuales contra el conteo de combiniaciones unicas de tarjeta mes_anio y nos deberia de dar lo mismo

In [99]:

```
variables_comercio['trans_por'].sum()
```

Out[99]:

```
600.0
```

In [100]:

```
len(variables_comercio.groupby(['tarjeta', 'mes_anio']))
```

Out[100]:

```
600
```

Para terminar de generar la variable convertimos filas que nos indicaban porcentualmente cuanto se gastaba en cada comercio a columnas para que la unidad de analisis entre la tabla base y la variable calculada sea la misma. Note que teniamos unos valores null en la columna comercio pero fueron ingnorados al utilizar la función de pivot_table

In [101]:

```
variables_comercio[variables_comercio['comercio'].isnull()]
```

Out[101]:

	tarjeta	mes_anio	comercio	conteo_comercio	conteo_mensual	trans_por
1280	46.0	2020-02-01	NaN	1	8	0.125000
1548	56.0	2020-04-01	NaN	1	7	0.142857
1602	58.0	2020-02-01	NaN	1	6	0.166667
2775	100.0	2020-01-01	NaN	1	11	0.090909

In [103]:

```
variables_comercio_2 = variables_comercio.pivot_table(index=['tarjeta', 'mes_anio'], colu
mns='comercio',values='trans_por',dropna=False).reset_index()
variables_comercio_2 = variables_comercio_2.fillna(0)

variables_comercio_2.head()
```

Out[103]:

comercio	tarjeta	mes_anio	Abarroterias	Comercio General	Electronicos	Farmacias	Gasolineras	Restaurantes	Ropa	Supermer
0	1.0	2020-01-01	0.000000	0.00	0.000000	0.166667	0.000000	0.166667	0.166667	0.5
1	1.0	2020-02-01	0.000000	0.00	0.000000	0.000000	0.200000	0.200000	0.000000	0.6

2	1.0	2020-03-01	0.111111	0.00	0.000000	0.000000	0.111111	0.444444	0.000000	0.2
comercio	tarjeta	mes_anio	Abarroterías	Comercio General	Electrónicos	Farmacias	Gasolineras	Restaurantes	Ropa	Supermer
3	1.0	2020-04-01	0.076923	0.00	0.076923	0.153846	0.307692	0.230769	0.000000	0.1
4	1.0	2020-05-01	0.000000	0.25	0.250000	0.000000	0.250000	0.000000	0.250000	0.0

12. Realice un join para "pegarle" las variables de comercio a su tabla base

In [104]:

```
base_vars = base_vars.merge(variables_comercio_2, on = ['tarjeta', 'mes_anio'], how='lef
t')
base_vars.head()
```

Out[104]:

	nombre	apellido	tarjeta	fecha_nacimiento	edad	mes_anio	conteo_mensual	sumatoria_mensual	promedio_monto
0	Rosa	Barrientos	1	1984-12-19	35.057534	2020-01-01	6.0	7415.41	123
1	Rosa	Barrientos	1	1984-12-19	35.142466	2020-02-01	5.0	9578.51	191
2	Rosa	Barrientos	1	1984-12-19	35.473973	2020-06-01	10.0	16552.09	165
3	Rosa	Barrientos	1	1984-12-19	35.389041	2020-05-01	4.0	6377.68	159
4	Rosa	Barrientos	1	1984-12-19	35.306849	2020-04-01	13.0	22509.28	173

In [105]:

```
len(base_vars[base_vars['Farmacias'].isnull()])
```

Out[105]:

```
60
```

In [106]:

```
base_vars= (
base_vars.fillna(value=0) )
```

In [107]:

```
len(base_vars[base_vars['Farmacias'].isnull()])
```

Out[107]:

```
0
```